

ゲームポートフォリオ

名前：贾 坤
所属：日本電子専門学校
ゲーム制作科

目次

01



タイトル：SnakeGame

開発環境：Unity

開発人数：1人

URL： <https://github.com/Barbatos-git/SnakeGame.git>

02



タイトル：ShootingGame

開発環境：Unity

開発人数：1人

URL： <https://github.com/Barbatos-git/ShootingGame.git>

03



タイトル：DeffenceGame

開発環境：Unity

開発人数：1人

URL： <https://github.com/Barbatos-git/DeffenceGame.git>

04



タイトル：CommandBattle

開発環境：Unity

開発人数：4人

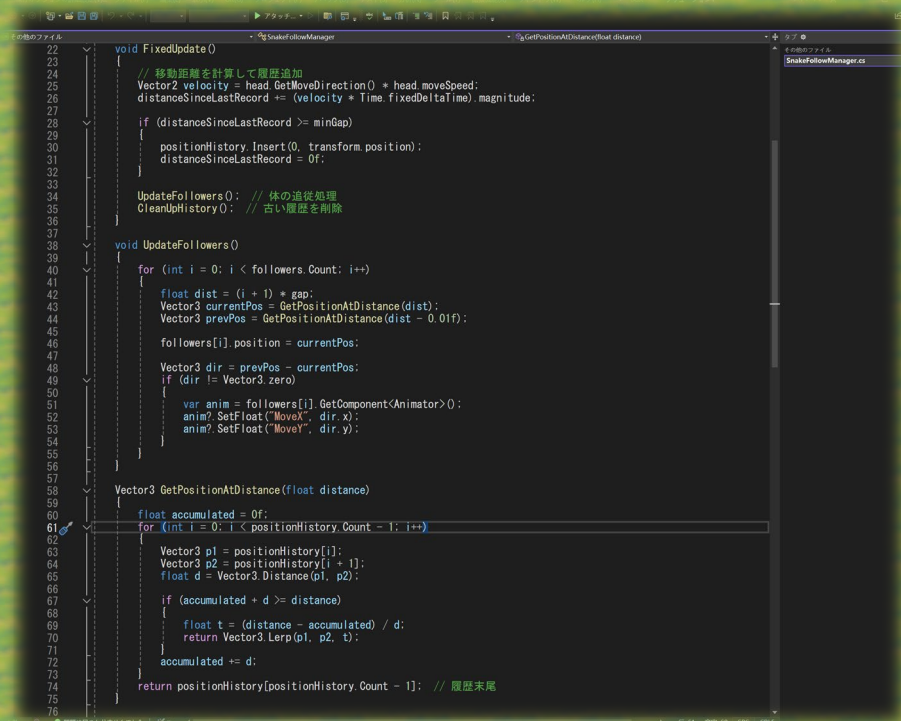
URL： <https://github.com/Barbatos-git/CommandBattle.git>



SnakeGame Unity 6 を使用して制作された 2D スネーク風ゲームです。

ゲーム開始時に男性または女性キャラクターを選択でき、キャラクターが肉を食べることで体が伸び、スピードもアップしていきます。

ゲーム終了後にはスコアと肉の取得数がアニメーション付きで表示されます。



1. 履歴ベースの追従アルゴリズム

蛇の移動経路を positionHistory に記録し、体の各パーツが一定距離ごとにその履歴を参照することで、スムーズな追従を実現しました。

フレームごとの位置ではなく 移動距離に応じて履歴を追加 する仕組みにしたため、速度変化や加速時でも自然な動きを維持できます。

2. 線形補間 (Lerp) による精密な位置計算

各パーツの位置を「履歴の2点間を線形補間」して算出することで、ガタつきのない滑らかな動きにしています。

これにより、単純なグリッドベースのスネークではなく 自然でキャラクター性のあるアニメーション を表現可能になりました。

3. 柔軟な履歴管理とパフォーマンス最適化

CleanUpHistory() で履歴の長さを動的に制御し、無駄なデータを削除。

蛇の長さが伸びてもメモリ負荷を抑え、長時間プレイでも快適に動作します。

4. アニメーション連動

各ボディパーツに Animator をアタッチし、追従時に「移動方向 (MoveX/MoveY)」を渡す仕組みを追加。

これにより キャラクターごとの動きや方向転換の表情 を自然に表現できました。



ShootingGame

Unityエンジンで制作された2Dシューティングゲームです。リアルタイムの攻撃とカード戦略を組み合わせたゲームで、プレイヤーは敵の弾幕を回避しながら敵を撃破し、最終的にボスとのカードバトルに挑みます。

敵を10体倒すごとにカードを入手（最大3枚）。ボスは初期から3枚カードを所持し、プレイヤーと「3枚カード」ルールで対戦。手札によって通常弾・竜巻・落雷の特殊弾が変化されます。

敵を倒すことでスコアと撃破数を蓄積、ボス撃破でボーナス加点。ゲーム終了後、撃破数・スコア・プレイ時間によって、ランク評価（SSS～D）を計算して表示します。

1. 直感的なカード操作（UI+入力連動）

プレイヤーはJ/K/Lキーでカードを選択し、もう一度押すことで出牌できる仕組みを実装します。

選択中のカードはUI上でハイライト（位置を持ち上げる）表現を追加し、視覚的にわかりやすくしました。

2. 手札・予備札管理の最適化

手札は最大3枚まで保持し、それ以上のカードは予備領域（ReserveArea）に送られる仕組みを導入します。

予備札は20秒後に自動消滅する仕組みを追加し、ゲームテンポを維持します。

TryFillHandReserve()により、カードを出した瞬間に予備札から自動補充される自然な流れを実現しました。

3. 拡張性を意識した設計

将来的に「ペア・ストレート・トリプル」などの役判定を柔軟に拡張できる設計にしました。

CardDataとCardFactoryを分離し、カード生成と表示をモジュール化します。

```
void HandleInput(int index)
{
    Debug.Log($"キー入力 index = {index}, 手札: {(handCardObjects[index] == null ? "空": "あり"}), data: {(handCardData[index] == null ? "空": "あり"});");
    if (!IsHandFull() || reserveCard == null)
    {
        return;
    }

    if (index < 0 || index >= maxHand) return;

    if (selectedIndex == index)
    {
        PlayCard(index);
        selectedIndex = -1;
    }
    else
    {
        selectedIndex = index;
        HighlightCard(index);
    }
}

void HighlightCard(int index)
{
    for (int i = 0; i < handCardObjects.Length; i++)
    {
        if (handCardObjects[i] == null) continue;
        var rt = handCardObjects[i].GetComponent<RectTransform>();
        if (rt != null)
        {
            rt.anchoredPosition = new Vector2(rt.anchoredPosition.x, (i == index ? 30f : 0f));
        }
    }
}

void ResetHighlight()
{
    for (int i = 0; i < handCardObjects.Length; i++)
    {
        if (handCardObjects[i] == null) continue;
        var rt = handCardObjects[i].GetComponent<RectTransform>();
        if (rt != null)
        {
            rt.anchoredPosition = new Vector2(rt.anchoredPosition.x, 0f);
        }
    }
}

public bool CanReceiveCard()
{
    return reserveCard == null && reserveCardObject == null;
}

public void AddCard(CardData data)
{
    if (!IsHandFull() && !CanReceiveCard())
    {
        return;
    }

    reserveCard = data;
    reserveCardObject = cardFactory.CreateCard(data, reserveArea, true, false);
    reserveCardObject.GetComponent<RectTransform>().anchoredPosition = Vector2.zero;

    var slot = reserveCardObject.AddComponent<ReserveCardSlot>();
    slot.OnExpired = s => {
        reserveCard = null;
        reserveCardObject = null;
        Destroy(s.gameObject);
    };
}

void PlayCard(int index)
{
    if (index < 0 || index >= maxHand || handCards[index] == null || handCardObjects[index] == null)
    {
        return;
    }

    CardData played = handCards[index];
    GameObject playedCard = handCardObjects[index];

    playedCard.transform.SetParent(bossReserveArea, false);
    var rt = playedCard.GetComponent<RectTransform>();
    if (rt != null)
    {
        rt.anchoredPosition = Vector2.zero;
        rt.localScale = Vector3.one;
        rt.localRotation = Quaternion.identity;
    }

    playedCard.GetComponent<CardDisplay>().Flip(true);

    handCardObjects[index] = null;
    handCards[index] = null;

    ResetHighlight();
    TryFillHandFromReserve();

    Debug.Log($"プレイヤーはカードを出す: " + played);
}

void TryFillHandFromReserve()
{
    if (reserveCard == null) return;
    for (int i = 0; i < maxHand; i++)
    {
        if (handCardObjects[i] == null)
        {
            AddCard(reserveCard);
            Destroy(reserveCardObject);
            reserveCard = null;
            reserveCardObject = null;
            break;
        }
    }
}

bool IsHandFull()
{
    for (int i = 0; i < maxHand; i++)
    {
        if (handCardObjects[i] == null || handCards[i] == null)
        {
            return false;
        }
    }
    return true;
}
```

```
void PlayCard(int index)
{
    if (index < 0 || index >= maxHand || handCards[index] == null || handCardObjects[index] == null)
    {
        return;
    }

    CardData played = handCards[index];
    GameObject playedCard = handCardObjects[index];

    playedCard.transform.SetParent(bossReserveArea, false);
    var rt = playedCard.GetComponent<RectTransform>();
    if (rt != null)
    {
        rt.anchoredPosition = Vector2.zero;
        rt.localScale = Vector3.one;
        rt.localRotation = Quaternion.identity;
    }

    playedCard.GetComponent<CardDisplay>().Flip(true);

    handCardObjects[index] = null;
    handCards[index] = null;

    ResetHighlight();
    TryFillHandFromReserve();

    Debug.Log($"プレイヤーはカードを出す: " + played);
}

void TryFillHandFromReserve()
{
    if (reserveCard == null) return;
    for (int i = 0; i < maxHand; i++)
    {
        if (handCardObjects[i] == null)
        {
            AddCard(reserveCard);
            Destroy(reserveCardObject);
            reserveCard = null;
            reserveCardObject = null;
            break;
        }
    }
}

bool IsHandFull()
{
    for (int i = 0; i < maxHand; i++)
    {
        if (handCardObjects[i] == null || handCards[i] == null)
        {
            return false;
        }
    }
    return true;
}
```

```
public void AddCard(CardData data)
{
    if (!IsHandFull() && !CanReceiveCard())
    {
        return;
    }

    Debug.Log($"AddCard() 呼び出される: {data.rank} of {data.suit}");
    for (int i = 0; i < maxHand; i++)
    {
        if (handCardObjects[i] == null)
        {
            handCards[i] = data;
            Debug.Log($"handCards[{i}] 代入完了");

            GameObject cardGO = cardFactory.CreateCard(data, handPositions[i], true, false);
            cardGO.transform.SetParent(handPositions[i], false);
            cardGO.GetComponent<RectTransform>().anchoredPosition = Vector2.zero;
            handCardObjects[i] = cardGO;

            Debug.Log($"handCardObjects[{i}] 代入完了 -> GameObject: {cardGO.name}");
            return;
        }
    }

    if (!CanReceiveCard())
    {
        reserveCard = data;
        reserveCardObject = cardFactory.CreateCard(data, reserveArea, true, false);
        reserveCardObject.GetComponent<RectTransform>().anchoredPosition = Vector2.zero;

        var slot = reserveCardObject.AddComponent<ReserveCardSlot>();
        slot.OnExpired = s => {
            reserveCard = null;
            reserveCardObject = null;
            Destroy(s.gameObject);
        };
    }
    else
    {
        return;
    }
}

void PlayCard(int index)
{
    if (index < 0 || index >= maxHand || handCards[index] == null || handCardObjects[index] == null)
    {
        return;
    }
}
```


SSS

KILLS: 33

Score: 713

Time: 00:44

1. カウントアップ演出で達成感を強調

Kills / Score を即時表示するのではなく、CountUp()コルーチンで 数値を0から加算 していくアニメーションを実装します。プレイヤーが「スコアが伸びていく感覚」を得られ、結果発表のワクワク感を演出します。

2. 効率評価によるランク判定

「スコア ÷ プレイ時間」の効率値を用いて S~Dランクを決定します。

プレイ時間が短く高スコアほど高ランクになり、ゲーム性と達成感が直結する仕組みです。

3. 高ランク専用のフラッシュ演出

S / SS / SSS ランクの場合、DOTweenを用いて 拡大+フェードインの特別演出 を追加します。

普通のランクとの差別化を行い、ハイスコアを達成した喜びを強調します。

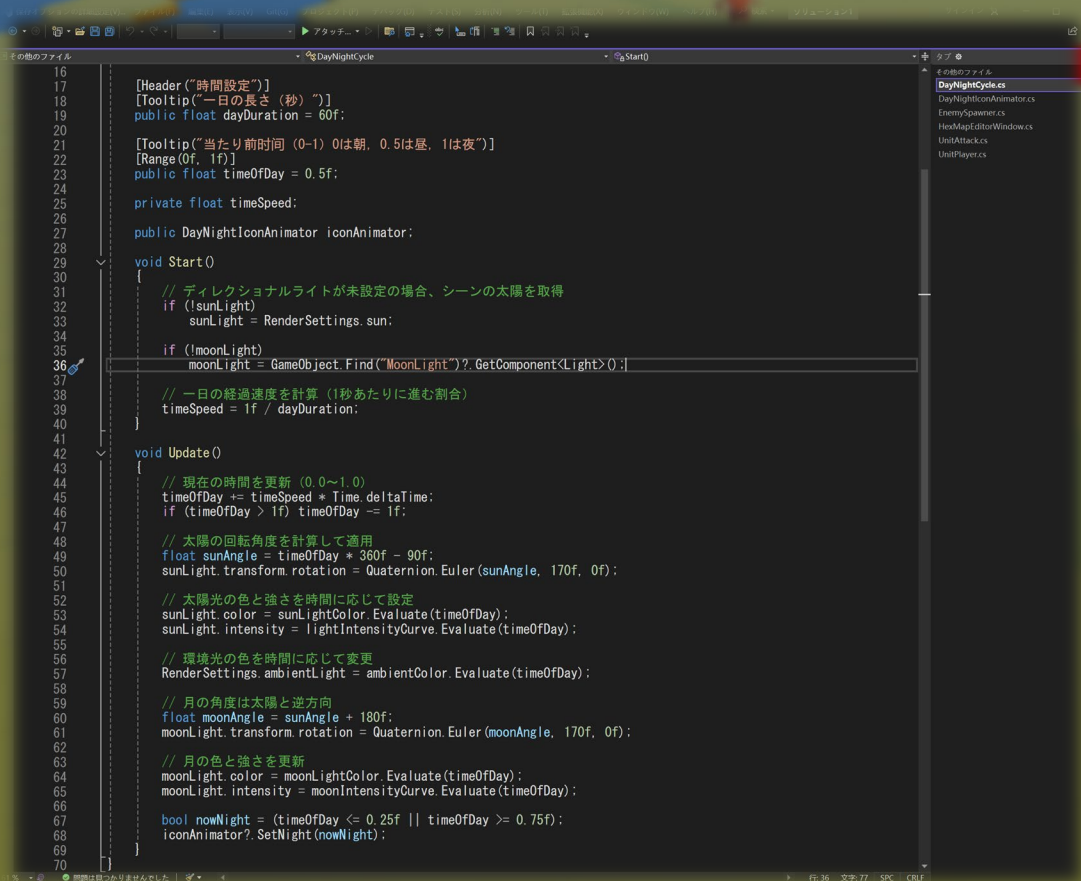


Deffence Game

Unity を使用して制作した 3D 防衛ゲームです。プレイヤーは兵士を配置し、昼夜が循環する中で夜ごとに強化される敵を迎え撃ちます。塔のHP管理や夜数の記録によって「どこまで耐えられるか」を競うゲーム性が強調されています。

3D マップや自動バトルAIに加え、塔破壊時の爆発・炎・煙のエフェクト、DOTweenによるUIアニメーション、AudioSourceを活用した3D音響演出によって、視覚・聴覚の両面から臨場感が演出されています。

タイトル画面、ゲーム本編、エンド画面という3シーン構成で遊びやすく設計されており、プレイから結果表示までの流れが明確に整理されています。



1. シーン側：昼夜サイクル

1日の進み方をパラメータ化：dayDuration と timeSpeed=1/dayDuration で速度を可変化。ゲームテンポに合わせて Inspector から即調整可能です。

時刻の循環処理：timeOfDay を 0～1 の範囲でループさせて、連続再生でも破綻しない設計です。

太陽の物理回転：timeOfDay * 360 - 90 の角度でディレクショナルライトを回転し、直感的な太陽運行を再現します。

色と強度を美術チューニング可能に：太陽・環境光・月光の Gradient/AnimationCurve を用意し、時間帯ごとの色温度や明るさを美術主導で調整できるようにしました。

月は太陽の反対位相：月の回転を太陽 + 180°で制御し、昼夜の見た目の整合性を確保します。

フォールバックに強い：RenderSettings.sun の自動取得や MoonLight の動的探索で、シーン設定漏れに強い初期化。

ゲーム統計と UI の連携：夜判定 (≤ 0.25 || ≥ 0.75) を一元で出し、UI アニメータへ通知するだけの疎結合設計。後続の演出差し替えが容易です。

2. UI側：太陽／月アイコンの交替演出

状態変化の重複抑止：isNight を用いた早期リターンで、同一状態への連続トリガーによる多重再生を防止します。

DOTween で滑らかな交替：CanvasGroup.DOFade と rotateRoot.DOLocalRotate を併用し、フェード＋回転で印象的かつ軽量の切替演出を実現。Ease も InOutSine で自然です。

回転ルートのリセット：昼へ戻るときは 360°回転＋OnComplete で角度を 0° に正規化し、累積誤差や角度オーバーフローを回避します。

日数カウントのタイミング設計：夜→昼へ切替時に GameStats.nightCount++ を入れており、「夜を越えた」達成感が UI と同期します。

見た目と当たり判定の分離：CanvasGroup で透明度制御しつつ SetActive(true) を併用、アニメ開始前の不意なチラつきを抑制します。

中心基準のレイアウト：rotateRoot を“回転の親”として用意し、アイコンの位置関係を崩さずに演出だけ差し替え可能です（将来の半径・軌道表現にも拡張しやすい）。



```
16 [MenuItem("Tools/Hex Map Editor (Advanced)")]
17 public static void Open()
18 {
19     GetWindow<HexMapEditorWindow>("Hex Map Editor");
20
21     private void OnEnable()
22     {
23         SceneView.duringSceneGui += OnSceneGUI;
24         RebuildTileMap();
25         ClearPreview();
26     }
27
28     private void OnDisable()
29     {
30         SceneView.duringSceneGui -= OnSceneGUI;
31         ClearPreview();
32     }
33
34     private void OnGUI()
35     {
36         GUILayout.Label("Tile Prefabs", EditorStyles.boldLabel);
37         grassPrefab = (GameObject)EditorGUILayout.ObjectField("Grass Prefab", grassPrefab, typeof(GameObject), false);
38         waterPrefab = (GameObject)EditorGUILayout.ObjectField("Water Prefab", waterPrefab, typeof(GameObject), false);
39         towerPrefab = (GameObject)EditorGUILayout.ObjectField("Tower Prefab", towerPrefab, typeof(GameObject), false);
40         selectedType = (HexTileType)EditorGUILayout.EnumPopup("Selected Type", selectedType);
41         hexRadius = EditorGUILayout.FloatField("Hex Radius", hexRadius);
42
43     private void OnSceneGUI(SceneView sceneView)
44     {
45         Event e = Event.current;
46         if (e == null || sceneView.camera == null) return;
47
48         Ray ray = HandleUtility.GUIPointToWorldRay(e.mousePosition);
49
50         if (Physics.Raycast(ray, out RaycastHit hit))
51         {
52             Vector3 worldPos = hit.point;
53             (int q, int r) = WorldToHex(worldPos);
54             Vector3 gridPos = HexToWorld(q, r);
55
56             UpdatePreview(gridPos);
57
58             if (e.type == EventType.MouseDown && e.button == 0)
59             {
60                 PlaceTile(q, r, gridPos);
61                 e.Use();
62             }
63             else if (e.type == EventType.MouseDown && e.button == 1)
64             {
65                 RemoveTile(q, r);
66                 e.Use();
67             }
68         }
69     }
70 }
```

```
136 void ClearPreview()
137 {
138     if (previewInstance) DestroyImmediate(previewInstance);
139 }
140
141 (int, int) WorldToHex(Vector3 pos)
142 {
143     float q = (Mathf.Sqrt(3f) / 3f * pos.x - 1f / 3f * pos.z) / hexRadius;
144     float r = (2f / 3f * pos.z) / hexRadius;
145     return CubeRound(q, r);
146 }
147
148 (int, int) CubeRound(float qf, float rf)
149 {
150     float x = qf;
151     float z = rf;
152     float y = -x - z;
153
154     int rx = Mathf.RoundToInt(x);
155     int ry = Mathf.RoundToInt(y);
156     int rz = Mathf.RoundToInt(z);
157
158     float dx = Mathf.Abs(rx - x);
159     float dy = Mathf.Abs(ry - y);
160     float dz = Mathf.Abs(rz - z);
161
162     if (dx > dy && dx > dz) rx = -ry - rz;
163     else if (dy > dz) ry = -rx - rz;
164     else rz = -rx - ry;
165
166     return (rx, rz);
167 }
168
169 Vector3 HexToWorld(int q, int r)
170 {
171     float x = hexRadius * Mathf.Sqrt(3f) * (q + r / 2f);
172     float z = hexRadius * 1.5f * r;
173     return new Vector3(x, 0, z);
174 }
175
176 private void RebuildTileMap()
177 {
178     tileMap.Clear();
179
180     root = GameObject.Find("HexMapRoot")?.transform;
181     if (root == null) return;
182
183     foreach (Transform child in root)
184     {
185         HexTile tile = child.GetComponent<HexTile>();
186         if (tile != null)
187         {
188             var key = (tile.q, tile.r);
189             if (tileMap.ContainsKey(key))
190                 tileMap[key] = child.gameObject;
191         }
192     }
193 }
```

1. 六角形マップエディタ

SceneView直操作&プレビュー：

SceneView.duringSceneGuiでレイキャストし、マウス下の六角グリッド位置を即座に算出。半透明のプレビューを表示して設置ミスを防止。左クリックで配置／右クリックで削除に統一。

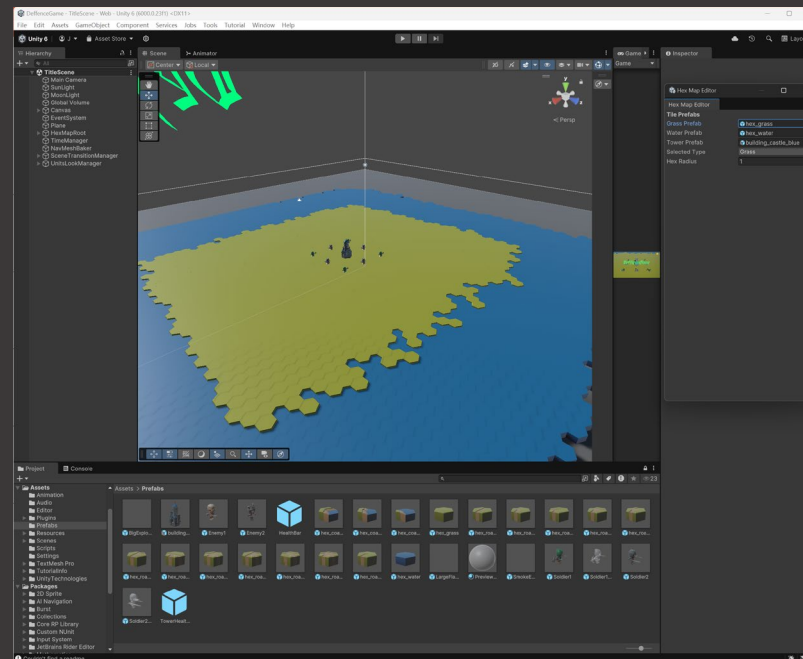
座標系の厳密化：ワールド座標→軸座標 (q,r) 変換でcube roundを実装し、誤差の少ない正確なヘックスナップを実現します。逆変換 (Hex→World) も用意して往復可能です。

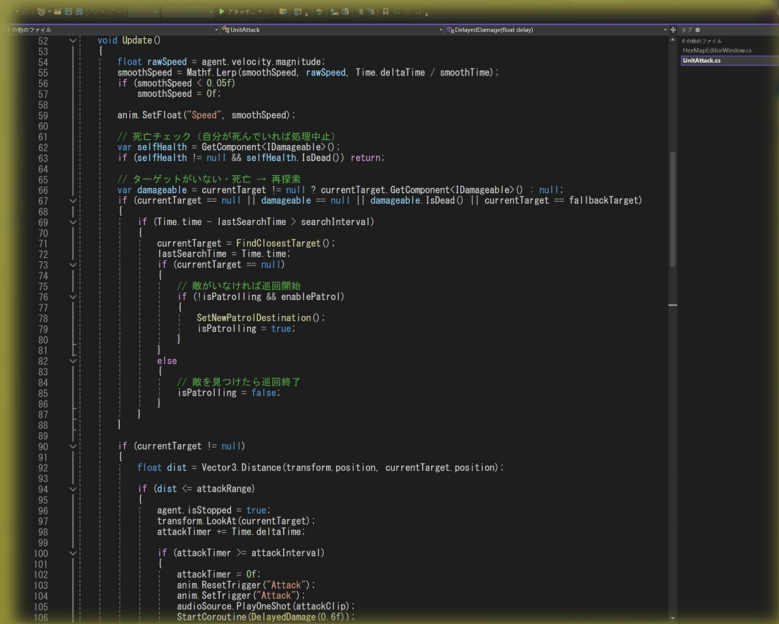
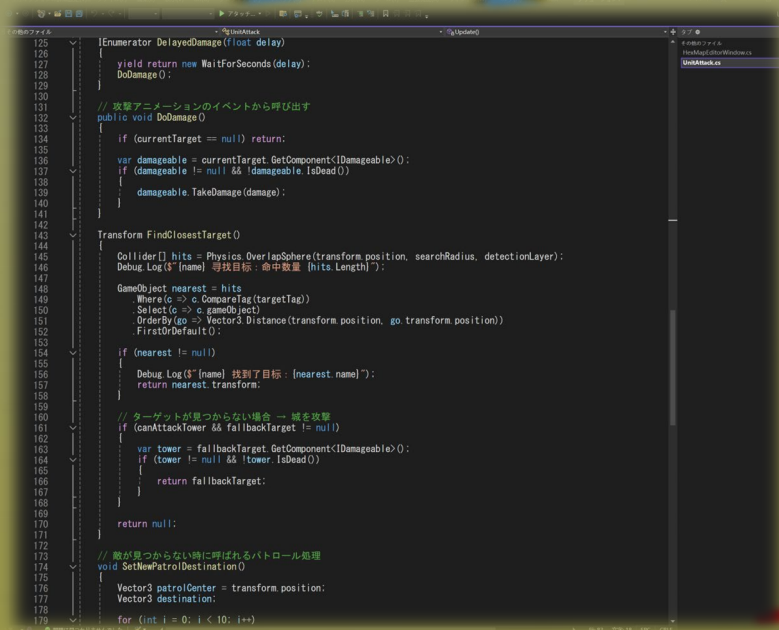
プレハブ接続と再構築：

PrefabUtility.InstantiatePrefabでプレハブ接続を維持し、HexMapRoot配下に配置。起動時はシーン内からタイルを走査して辞書を再構築し、途中保存や再読込に強いです。

柔軟なタイル管理：(q,r)をキーにしたDictionaryで一意管理。草／水／塔の列挙型選択とhexRadiusにより、アートやサイズ変更のチューニングが容易です。

```
79 void PlaceTile(int q, int r, Vector3 pos)
80 {
81     if (!root) root = GameObject.Find("HexMapRoot")?.transform;
82     if (!root) return;
83
84     GameObject go = new GameObject("HexMapRoot");
85     root = go.transform;
86
87     if (tileMap.ContainsKey((q, r)))
88     {
89         DestroyImmediate(tileMap[(q, r)]);
90     }
91
92     GameObject prefab = selectedType switch
93     {
94         HexTileType.Grass => grassPrefab,
95         HexTileType.Water => waterPrefab,
96         HexTileType.Tower => towerPrefab,
97         _ => null;
98     };
99
100     if (prefab == null) return;
101
102     GameObject tile = (GameObject)PrefabUtility.InstantiatePrefab(prefab);
103     tile.transform.position = pos;
104     tile.transform.SetParent(root);
105     var hex = tile.GetComponent<HexTile>();
106     if (!hex) hex = tile.AddComponent<HexTile>();
107     hex.Init(q, r, selectedType);
108     tileMap[(q, r)] = tile;
109
110     void RemoveTile(int q, int r)
111     {
112         if (tileMap.TryGetValue((q, r), out GameObject tile))
113         {
114             DestroyImmediate(tile);
115             tileMap.Remove((q, r));
116         }
117     }
118
119     void UpdatePreview(Vector3 pos)
120     {
121         if (!previewInstance)
122         {
123             previewInstance = GameObject.CreatePrimitive(PrimitiveType.Cylinder);
124             previewInstance.name = "PreviewTile";
125             previewInstance.GetComponent<Renderer>().sharedMaterial = new Material(Shader.Find("Transparent/Diffuse"));
126             previewInstance.GetComponent<Collider>().enabled = false;
127
128             previewInstance.transform.position = pos + Vector3.up * 0.01f;
129             previewInstance.transform.localScale = Vector3.one * hexRadius * 1.8f;
130         }
131     }
132 }
```





2. プレイヤー／敵 共通の索敵・攻撃 AI

負荷と堅牢性への配慮：searchIntervalで索敵頻度を制御し、IDamageable.IsDead()チェックで死亡対象を即スキップ。自分が死亡したら早期リターンで無駄処理を抑制します。

最短ターゲット選択の明確化：

Physics.OverlapSphere→tagフィルタ→距離
OrderByで最も近い敵を決定。見つからない場合は塔へフォールバック。

移動／停止と攻撃の状態遷移がシンプル：射程内ならagent.isStopped = true + LookAtで攻撃、外ならSetDestinationで追跡。遅延ダメージ（コルーチン）やアニメイベント呼出しに対応します。

待機時のパトロール：敵不在時はNavMesh上で半径内ランダム目的地をサンプリングして巡回、到達したら次点を設定。スタック回避や距離しきい値も実装します。

表現面の細部：移動速度をスムージングしてAnimatorのSpeedに反映、攻撃SEはPlayOneShotで簡潔に再生。Gizmosで攻撃範囲／索敵範囲を可視化してデバッグ性を向上します。

CommandBattle

(チーム制作)



Unity を使用して制作した2人対戦型のシミュレーション（戦棋）ゲームです。各プレイヤーは5体から3体を選び、配置→バトルの順で進行。各ターンで1体が「移動→攻撃/スキル/待機」を実行し、どちらかのチームが全滅すると決着します。

マップ上の位置取りと多様な攻撃範囲で戦略を組み立て、特定ユニットはQTEでダメージが変動します（弓/戦士＝連打、魔法＝順番入力、アサシン＝素早い3回入力）。

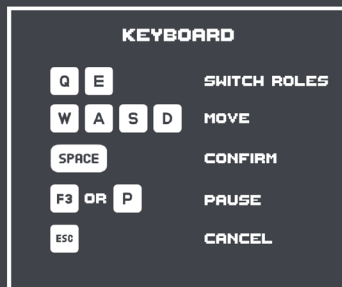
キーボード／ゲームパッド対応のローカル対戦。DOTweenによるフェードやダメージ表示などのビジュアル効果で、読み合いが気持ちよく回るテンポ感を実現します。

担当

リーダーとしてだけでなく、メインプログラマーとしても活動し、TileMapの作成、キャラクターの交互動作（マップ上での移動・攻撃）、ターン制御の実装、

キャラクター選択画面のロジックや配置設定の演出、ゲーム画面への引き継ぎによるキャラクター生成、

ダメージ数値の表示、最後の結合などを担当しました。



P1: XBOX CONTROLLER

SUCCESS



P2: KEYBOARD

SUCCESS

“最初の押下 = P1、次の押下 = P2”の自動割当て：Input System の全デバイスを走査し、ButtonControl.wasPressedThisFrameで即時バインドします。

二重割当ての防止：P1/P2のdeviceを相互チェックし、同一デバイスが両者に割り当たらないよう保護します。

入力方式の自動判別：device is GamepadでGamepad/Keyboardを切替えてcontrolSchemeを自動設定します。

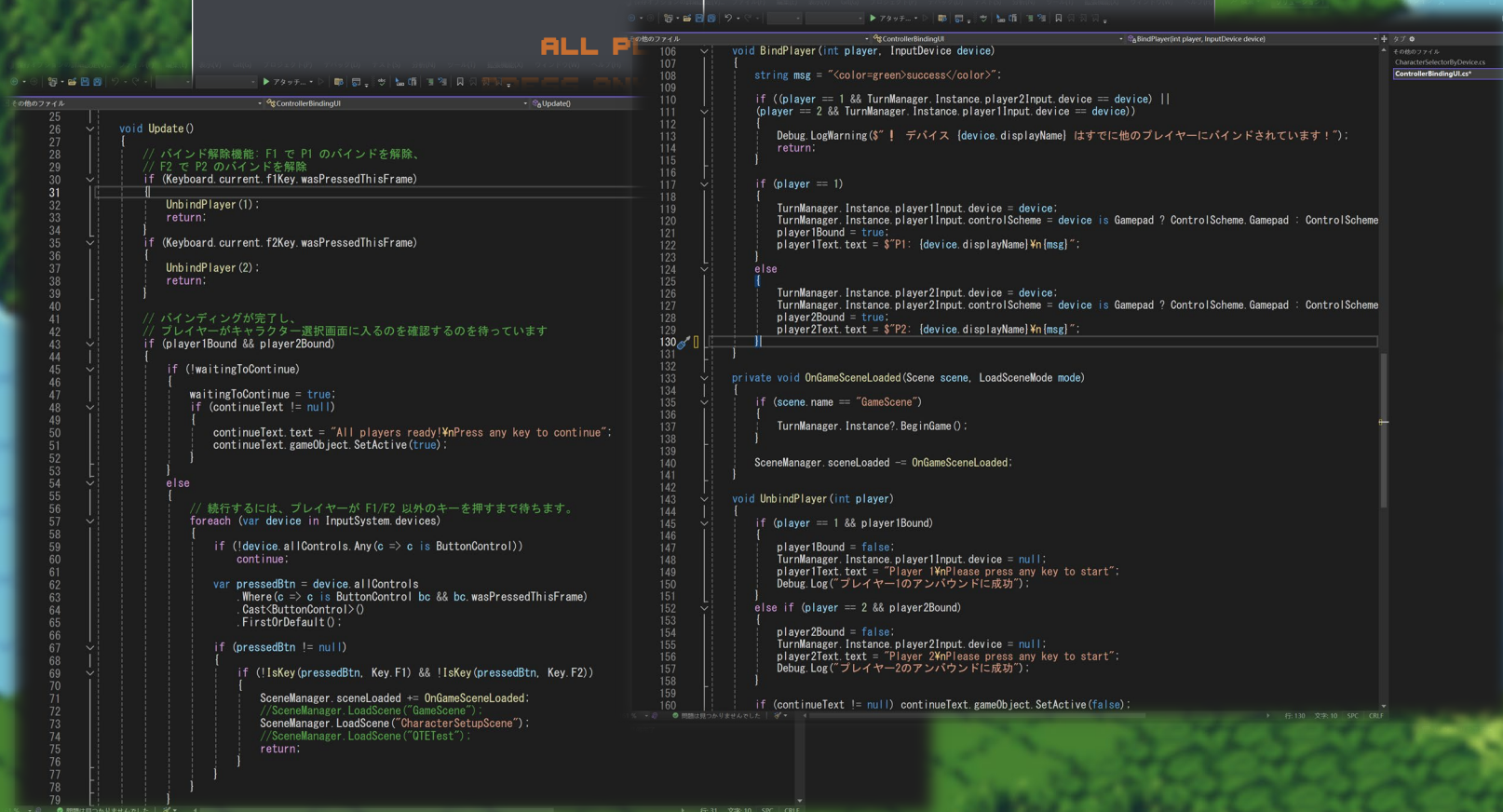
アンバインドの運用性：F1（P1）/F2（P2）で解除・初期文言に戻す・continueTextを隠すまでを一括で実行します。

誤操作対策付きの遷移待ち：両者準備完了で「All players ready!」表示→F1/F2以外の押下を検出して次シーンへ。

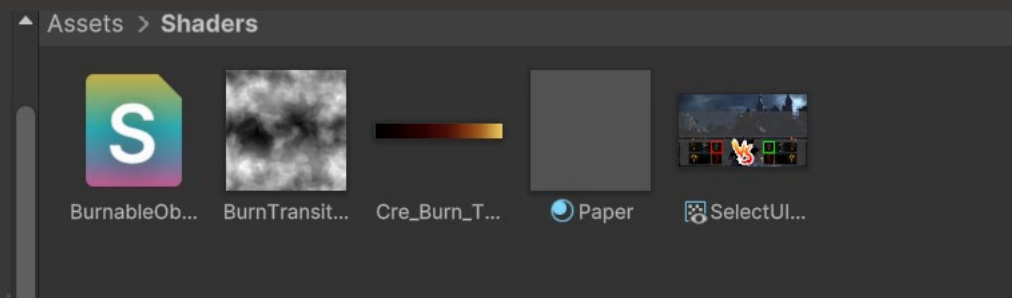
二重実行防止フラグ：waitingToContinueで遷移待ちメッセージの多重表示／処理を抑止します。

安全なシーン遷移フック：sceneLoadedでGameScene時のみBeginGame()を呼ぶ設計です。

UIの即時フィードバック：TMPで成功を緑色表示、初期プロンプトも明確化し、状態が視覚的に分かります。



ADF: 20
MOV: 4



1. 紙の燃焼トランジション

- ・共有マテリアルの汚染回避：new Material(burnMaterial)でインスタンス化し、UIのRawImage.materialに割当て。以後 _MaskThreshold をカラーチンで0→1へ滑らかに更新。終了時は非表示&コールバックで次処理へ接続します。

- ・再利用しやすいAPI：StartBurn(Action onFinished)/ResetBurn()で、どの画面遷移にも使い回せる汎用コントローラに。

2. キャラ選択とプレビュー

- ・グリッド操作の“鏡像”最適化：P1は左右入力を反転し、互いの視点で直感的にカーソルが動くUIに。列数columns指定&**行列包絡（ラップ）**で端でも自然に移動します。

- ・ホバープレビュー→確定昇格：スロットに半透明プレハブを一時表示し、確定時にそのまま正式インスタンス化（番号テキストも付与）。キャンセル時は最後尾のみ安全に巻き戻し。

- ・ロック&演出の分離：各自が3体選んだ後、Confirmでロック→燃焼演出StartBurn()完了時に選択UIを閉じて布陣システムを起動します。状態はplayer1Locked/2LockedとdeployedStartedで多重実行を防止します。

- ・データの橋渡し：選択結果をPlayerDeployDataへ転送し、次フェーズで正確に復元できるように単純なList<int>で保持します。

3. 派兵・布陣

- ・セル占有の二重チェック：両陣営の配置リストを横断して同一セルを拒否し、衝突を未然に防止します。Tilemap上の中心座標に実体を生成します。
- ・段階的ロックとUndo：各プレイヤーは1体ずつ確定し、Cancelで直近のみ撤回（実体破棄＋配列から除去）。3体配置後にConfirmでロック、両者ロックで短い遅延後にGameSceneへ遷移します。
- ・初期向きの表現整備：生成直後にAnimatorのDirectionを陣営別に設定し、戦闘開始時の視覚的一貫性を担保します。



```
public void RegisterUnit(UnitActionHandler unit)
{
    Vector3Int cell = unit.unitController.GetCurrentCell();
    unitMap[cell] = unit;
    //if (!allUnits.Contains(unit)) allUnits.Add(unit);
    if (!allUnits.Exists(u => u.uniqueID == unit.uniqueID))
    {
        allUnits.Add(unit);
    }
}

public void UnregisterUnit(UnitActionHandler unit)
{
    Vector3Int cell = unit.unitController.GetCurrentCell();
    if (unitMap.ContainsKey(cell) && unitMap[cell] == unit)
    {
        unitMap.Remove(cell);
        allUnits.Remove(unit);
    }
}

public UnitActionHandler GetUnitAtCell(Vector3Int cell)
{
    unitMap ??= new Dictionary<Vector3Int, UnitActionHandler>();
    foreach (var unit in allUnits)
    {
        if (!unit.isActiveAndEnabled) continue;
        Vector3Int pos = GridSelector.Instance.tilemap.WorldToCell(unit.transform.position);
        if (pos == cell) return unit;
    }
    return null;
}

public void EndTurn()
{
    currentTurn = (currentTurn == UnitOwner.Player1) ? UnitOwner.Player2 : UnitOwner.Player1;
    Debug.Log($"Turn changed to {currentTurn}");
    currentInputDevice = GetInputDeviceForPlayer(currentTurn);

    // すべてのユニットの状態をリセット
    foreach (var unit in FindObjectsOfType<UnitActionHandler>())
    {
        if (unit.owner == currentTurn)
        {
            unit.hasMoved = false;
            unit.hasActed = false;
        }
    }

    TurnBannerUI.Instance.ShowTurn(currentTurn);
    //プレイヤーのユニット選択開始
    UnitSelectorManager.Instance.StartTurn(currentTurn);
}

void AssignDevicesAutomatically()
```

P1LOCKED

4. ターン制御

- ・早期初期化&永続化：[DefaultExecutionOrder(-100)]で最速起動し、シングルトン＋DontDestroyOnLoadでシーンを跨いでも状態を保持します。CharaDataのInit()もここで実行して依存順を安定化します。
- ・ユニット管理の二層構造：unitMap (cell→unit)とallUnitsの併用。Register/Unregisterで登録を一元化し、uniqueID重複を排除して二重登録を防止します。
- ・堅牢なセル検索：GetUnitAtCellは辞書に依存せず全ユニットを走査し、tilemap.WorldToCell()でその場の実座標から特定。移動直後など辞書が古くても正しい結果を返す設計です。
- ・ターン遷移の責務分離：EndTurn()で手番をトグル→TurnBannerUI.ShowTurn()→UnitSelectorManager.StartTurn()の順に通知します。表示と操作のレイヤーを疎結合に分離します。
- ・入力デバイスの自動割当：ゲームパッド2台／1台／なしの各ケースを判別し、InputSystem.EnableDevice()まで含めて自動設定します。手番ごとにcurrentInputDeviceを更新して操作系を切替えます。
- ・フェアスタート：BeginGame()で全ユニットを再スキャンしてallUnitsを確実に構築し、コイントスで先攻をランダム決定します。開始演出と入力元の同期もここで一括です。
- ・拡張フックの用意：ユニットのhasMoved/hasActedリセット処理はコメントで残し、将来の“ターン開始時初期化”に備えた差し込みポイントを明示します。

1. 移動

・“行ける／行けない”を色で即可視化：マンハッタン距離でvalidCellsを作成し、水・岩タイルを判定して別レイヤーで表示します。味方や敵がいるマスは視覚的には提示しつつ、確定時のチェックで侵入を防ぐ“ソフトブロック”にして操作感を損ねません。

・A*と連動したライブ経路描画：選択セルが有効なら即座にPathfinding.FindPathで経路を算出・表示します。開始セルが欠けていれば先頭に差し込み、ユーザーに“迎る道”を常に見せます。

・操作の誤爆抑止：アナログ閾値＋クールダウンでカーソル暴走を防止します。キャンセル時は元位置へ吸着し、経路表示もクリアして誤操作からの復帰が速いです。

2. 経路探索

・占有回避付きA*：A*で、ゴール以外の占有セルを回避して経路を再構成。validCellsとtilemap.HasTileの二重条件で堅牢性を担保します。最小ヒープ簡易実装で依存を減らし、読みやすさも確保します。

3. 攻撃

・方向依存パターンを汎用化：攻撃者セル→選択セルから方向インデックスを求め、攻撃パターンのオフセットを回転します。どのスキル形状でも“向き”に追従できる拡張性を確保します。

・QTE×リソース連動のダメージ設計：QTE結果（Failed/Good/Great）に応じて**MP/STA消費量と威力（Low/Normal/Crit）**を連動します。リソース不足時は段階的にランクを落とす処理で、完全不発を避けつつゲームテンポを維持します。重攻撃/軽攻撃アニメの自動切替も内包です。

・UI/ターン制との疎結合：入力一時無効化→範囲可視化→確定後はコルーチンで演出完了待ち→NotifyUnitActedThisTurn()→EndTurn()という順序で演出と手番遷移を明確分離します。デバッグ用Gizmosで角度可視化も実装します。

```
var input = TurnManager.Instance.GetCurrentPlayerInput();
Vector2 moveInput = input.GetMoveInput();
Vector3Int dir = Vector3Int.zero;

if (moveInput.magnitude > 0.5f && Time.time - lastMoveTime > moveCooldown)
{
    if (Mathf.Abs(moveInput.x) > Mathf.Abs(moveInput.y))
        dir = moveInput.x > 0 ? Vector3Int.right : Vector3Int.left;
    else
        dir = moveInput.y > 0 ? Vector3Int.up : Vector3Int.down;

    if (dir != Vector3Int.zero)
    {
        lastMoveTime = Time.time;

        if (dir != Vector3Int.zero)
            MoveSelector(dir);

        // 角度値の調整
        Vector3 from = currentUnit.unitController.transform.position;
        Vector3 to = tilemap.GetCellCenterWorld(currentCell);
        Vector3 radDir = (to - from).normalized;
        Debug.DrawRay(from, radDir, Color.green);

        // Aボタン攻撃
        // 攻撃範囲エリアのリアルタイム表示（現在選択されているグリッドを中心）
        currentUnit.rangeVisualizer.Clear();
        currentUnit.rangeVisualizer.ShowCells(validCells, RangeVisualizer.Instance.attackTile);

        var attackPattern = currentUnit.GetAttackPattern();
        Vector3Int attackerCell = currentUnit.unitController.GetCurrentCell();
        Vector3Int selected = currentCell;

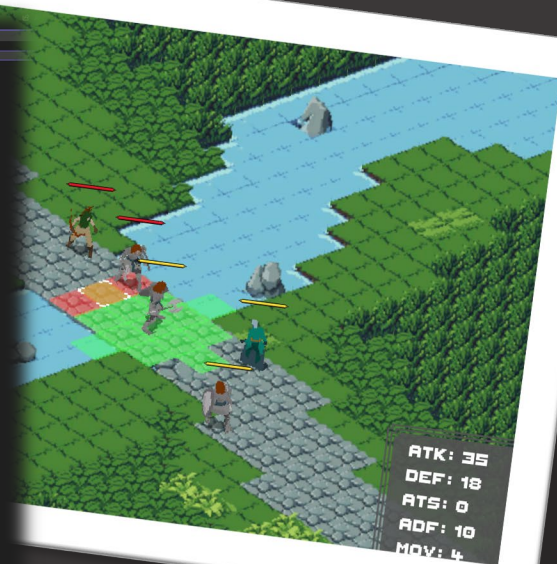
        int directionIndex = AttackDirectionHelper.GetDirectionIndex(attackerCell, selected, tilemap);

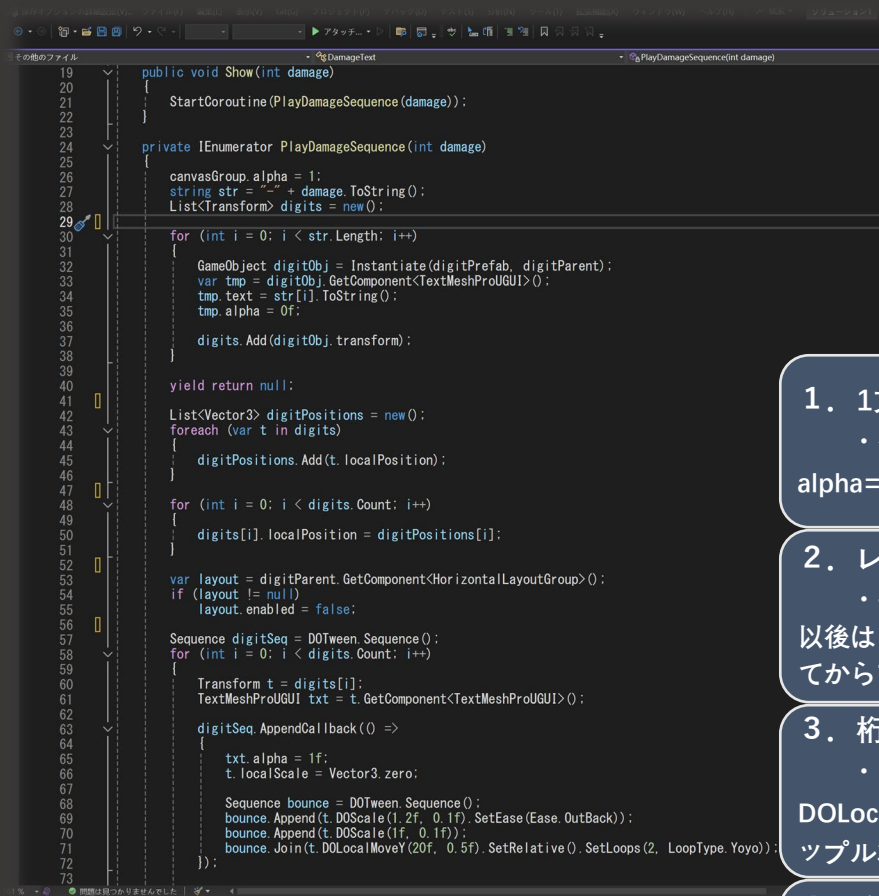
        Vector3Int[] baseOffsets = attackPattern != null ? attackPattern.offsets : new Vector3Int[] { Vector3Int.zero };
        Vector3Int[] finalOffsets = (attackPattern != null && attackPattern.useDirectionalOffset)
            ? AttackDirectionHelper.RotateOffsets(baseOffsets, directionIndex)
            : baseOffsets;

        currentUnit.rangeVisualizer.ShowPatternCells(currentCell, finalOffsets, RangeVisualizer.Instance.blockedTile);

        if (input.IsConfirmPressed())
        {
            if (validCells.Contains(currentCell))
            {
                if (!HasEnemyInArea(currentCell, finalOffsets))
                {
                    return;
                }
            }

            ExecuteAreaAttack(currentCell, finalOffsets);
        }
    }
}
```





1. 1文字ずつの生成 & 個別演出

・ダメージ値を文字列化（先頭に“-”付与）し、各桁を digitPrefab（TMP）から個別生成します。最初に alpha=0 で仕込み、桁ごとに表示演出を適用できる設計です。

2. レイアウト確定 → 位置スナップショット → レイアウト無効化

・一度 yield return null で HorizontalLayoutGroup による整列を完了させ、各桁の localPosition を記録します。以後は layout.enabled=false でレイアウト影響を切り、スナップショットした座標に再配置しても崩れないようにしてからアニメを行います。これで「整列の見やすさ」と「個別アニメ」を両立します。

3. 桁ごとの“ポップ + バウンス”を段階的に

・DOTween の Sequence を使い、桁ごとに AppendCallback → DOScale(1.2→1.0) のバウンス、DOLocalMoveY(20).SetRelative().SetLoops(2, Yoyo) の上下揺れを設定します。AppendInterval(0.08f) で**段階的（リップル状）**に出現させ、視認性と気持ちよさを両立します。

4. 全体の余韻演出とクリーンアップ

・桁アニメ後に 0.5秒の間を置き、テキスト全体を少し上へ漂わせつつ（transform.DOLocalMoveY(1.5f）CanvasGroup.DOFade(0,1f) でフェードアウト。OnComplete()==>Destroy(gameObject)) で自動破棄し、メモリ・ヒエラルキーをクリーンに保つ。

5. 使い勝手と堅牢性

・Show(int damage) からコルーチンで完結です。CanvasGroup による一括フェードで UI レイヤと無干渉、HorizontalLayoutGroup が無い場合も安全に動作します。将来の桁間隔調整用プロパティ（digitSpacing）も用意しており、拡張を想定した構造です。

最後までご覧いただき、ありがとうございました。